

Reviewer Pack — Minimal Rank of Tropical Curves Bounds DPLL Search Tree Heig...

Entry 958d4ff6720f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 13:32:06 UTC

Minimal Rank of Tropical Curves Bounds DPLL Search Tree Height

Entry ID: 958d4ff6720f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 13:32:06 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Resolution proof complexity

Statement:

For every satisfiable 3-CNF φ with m clauses on n variables, the minimal rank $r(\varphi)$ of its associated tropical curve is a lower bound on the height $h(\varphi)$ of the DPLL search tree for φ , such that $h(\varphi) \geq r(\varphi)$.

Rationale (proposer's reasoning):

Tropical geometry provides a geometric interpretation of Boolean functions, and its invariants have been used to analyze complexity. The minimal rank could capture essential features of the structure of the 3-CNF that influence the DPLL search tree's height.

Taxonomy category: c003b_cumulative_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b406af993e0ba7ac

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For every satisfiable 3-CNF φ with n variables ($n \leq 40$), if the minimal rank of its associated tropical curve is $r(\varphi)$, then the DPLL search tree height $h(\varphi)$ must satisfy $h(\varphi) \geq r(\varphi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	1.00	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "DPLL search tree" AND "minimal rank" - "resolution proof complexity" AND "tropical curve" AND "DPLL height bound" - "3-CNF satisfiability" AND "minimal rank tropical curve" AND "DPLL tree height"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=0, elapsed=0.4s

5.1 Generated Python source

```
#!/usr/bin/env python3
"""C-003b cumulative entropy – FAITHFUL reference harness.

Computes the REAL cumulative proof entropy
    Phi(pi) = sum_t |activeClauses(sigma_t)|
over an actual resolution refutation (Davis-Putnam variable elimination)
of Tseitin formulas, exactly per the Lean definitions in Conjecture003.lean:
    proofStateEntropy(sigma) := sigma.activeClauses.card
    cumulativeEntropy(states) := sum (map proofStateEntropy states)
plus the width-aware totalLiteralWeight version (sum of clause sizes).

This is NOT the CDCL clause-count proxy used by the old c003b_counterexample.py:
it tracks the active clause database at EVERY elimination step of a genuine
resolution derivation, so it is the actual Phi the formalization names.

Pure stdlib (no networkx/pysat), so it runs in the pipeline sandbox.
Protocol: run_trial(seed)->dict with TRIAL/RESULT lines.

Author: Ludovico Kubler (harness by the SEC engine, hand-verified).
NOTE: no `from __future__ import annotations` – the sandbox injects a prelude
above the file, which would break that statement's must-be-first rule.
"""

import json
import math
import random
import sys

Clause = frozenset # a clause = frozenset of signed ints (var or -var)

# ----- Tseitin formula generation -----

def random_regular_graph(n: int, degree: int, rng: random.Random):
    """Simple random regular graph via configuration model with retries."""
    if (n * degree) % 2 != 0:
```

```

    n += 1
    for _ in range(200):
        stubs = []
        for v in range(n):
            stubs += [v] * degree
        rng.shuffle(stubs)
        edges = set()
        ok = True
        for i in range(0, len(stubs), 2):
            u, w = stubs[i], stubs[i + 1]
            if u == w or (min(u, w), max(u, w)) in edges:
                ok = False
                break
            edges.add((min(u, w), max(u, w)))
        if ok and len(edges) == n * degree // 2:
            return n, sorted(edges)
    # fallback: cycle + chords
    edges = {(i, (i + 1) % n) for i in range(n)}
    return n, sorted((min(a, b), max(a, b)) for a, b in edges)

def tseitin_cnf(n: int, edges: list, charge: dict) -> list:
    """Tseitin CNF over edge variables (1-indexed). For each vertex, clauses
    enforce parity of incident edges = charge[v]."""
    evar = {}
    for i, (u, v) in enumerate(edges):
        evar[(u, v)] = i + 1
        evar[(v, u)] = i + 1
    inc = {v: [] for v in range(n)}
    for (u, v) in edges:
        inc[u].append(evar[(u, v)])
        inc[v].append(evar[(u, v)])
    clauses = []
    for v in range(n):
        vars_ = inc[v]
        k = len(vars_)
        tgt = charge.get(v, 0)
        for mask in range(1 << k):
            if bin(mask).count("1") % 2 != tgt:
                cl = []
                for j, var in enumerate(vars_):
                    cl.append(-var if (mask >> j) & 1 else var)
                clauses.append(frozenset(cl))
    return clauses

# ----- Resolution by Davis-Putnam variable elimination -----

def resolve(c1: frozenset, c2: frozenset, var: int):
    """Resolvent of c1, c2 on var (c1 has +var, c2 has -var). None if tautology."""
    r = (c1 - {var}) | (c2 - {-var})
    for lit in r:
        if -lit in r:
            return None # tautology
    return frozenset(r)

```

```

def dp_refutation_phi(clauses: list, rng: random.Random):
    """Run DP elimination in min-occurrence order; track the active clause DB
    after each elimination. Returns (phi_count, phi_weight, n_steps, derived_empty).

    phi_count = sum_t |DB_t| (proofStateEntropy = card)
    phi_weight = sum_t sum_{C in DB_t} |C| (totalLiteralWeight version)
    """
    db = set(clauses)
    variables = set(abs(l) for c in db for l in c)
    phi_count = len(db)
    phi_weight = sum(len(c) for c in db)
    n_steps = 1
    derived_empty = frozenset() in db
    MAX_DB = 200000 # guard against blow-up on bad instances

    while variables and not derived_empty:
        # min-occurrence elimination order (standard good heuristic)
        occ = {v: 0 for v in variables}
        for c in db:
            for l in c:
                if abs(l) in occ:
                    occ[abs(l)] += 1
        var = min(variables, key=lambda v: occ[v])
        variables.discard(var)

        pos = [c for c in db if var in c]
        neg = [c for c in db if -var in c]
        others = [c for c in db if var not in c and -var not in c]

        resolvents = set()
        for cp in pos:
            for cn in neg:
                r = resolve(cp, cn, var)
                if r is not None:
                    resolvents.add(r)
                    if len(r) == 0:
                        derived_empty = True
        new_db = set(others) | resolvents
        # forward subsumption (keep minimal clauses) – cheap pass
        db = new_db
        phi_count += len(db)
        phi_weight += sum(len(c) for c in db)
        n_steps += 1
        if len(db) > MAX_DB:
            break

    return phi_count, phi_weight, n_steps, derived_empty

# ----- Separator (cheap balanced-ish) -----

def approx_separator_size(n: int, edges: list) -> int:
    """Cheap proxy for balanced separator size: min vertices whose removal
    splits the graph ~in half. We use a simple BFS-layer cut."""
    adj = {v: set() for v in range(n)}

```

```

for (u, v) in edges:
    adj[u].add(v); adj[v].add(u)
# BFS from 0, cut at the layer crossing the median
from collections import deque
seen = {0}; layer = {0: 0}; q = deque([0])
while q:
    x = q.popleft()
    for y in adj[x]:
        if y not in seen:
            seen.add(y); layer[y] = layer[x] + 1; q.append(y)
if len(seen) < n: # disconnected
    return 0
half = n // 2
# vertices on the boundary between the first ~half and the rest
order = sorted(range(n), key=lambda v: layer.get(v, 0))
side_a = set(order[:half])
cut = set()
for (u, v) in edges:
    if (u in side_a) != (v in side_a):
        cut.add(u if u not in side_a else v)
return len(cut)

```

----- Trial protocol -----

```

def run_trial(seed: int) -> dict:
    rng = random.Random(seed)
    # sweep several small sizes; report the scaling exponent of Phi vs n
    ns = [6, 8, 10, 12, 14]
    data = []
    for n0 in ns:
        n, edges = random_regular_graph(n0, 3, rng)
        charge = {v: 0 for v in range(n)}
        charge[0] = 1 # odd total -> UNSAT
        clauses = tseitin_cnf(n, edges, charge)
        phi_c, phi_w, steps, empty = dp_refutation_phi(clauses, rng)
        sep = approx_separator_size(n, edges)
        data.append({"n": n, "m_edges": len(edges), "sep": sep,
                    "phi_count": phi_c, "phi_weight": phi_w,
                    "steps": steps, "derived_empty": empty})
    # scaling: log-log slope of phi_count vs n
    xs = [math.log(d["n"]) for d in data if d["phi_count"] > 0]
    ys = [math.log(d["phi_count"]) for d in data if d["phi_count"] > 0]
    if len(xs) >= 2:
        mx = sum(xs) / len(xs); my = sum(ys) / len(ys)
        num = sum((x - mx) * (y - my) for x, y in zip(xs, ys))
        den = sum((x - mx) ** 2 for x in xs)
        slope = num / den if den else 0.0
    else:
        slope = 0.0
    biggest = data[-1]
    # Sub-conjecture under test (SC1): Phi >= sep^2 on the largest instance
    # (a concrete, falsifiable scaling claim derived from the C-003b thesis
    # that cumulative entropy is forced by the separator).
    holds = biggest["phi_count"] >= max(1, biggest["sep"]) ** 2
    return {

```

```

    "metric_name": "phi_count_loglog_slope_vs_n",
    "metric_value": round(slope, 4),
    "instances_tested": len(ns),
    "n_max": max(ns),
    "conjecture_holds": holds,
    "counterexample": "" if holds else
        f"n={biggest['n']} sep={biggest['sep']} phi={biggest['phi_count']} < sep^2",
    "detail": data,
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [11, 23, 37]
    results = []
    for s in seeds:
        r = run_trial(s)
        results.append(r)
        print("TRIAL: " + json.dumps(r))
    slopes = [r["metric_value"] for r in results]
    holds = sum(1 for r in results if r["conjecture_holds"])
    mean = sum(slopes) / len(slopes)
    if holds == len(results):
        print(f"RESULT: SUPPORTED mean_slope={mean:.4f} support_fraction=1.0")
    elif holds == 0:
        print(f"RESULT: FALSIFIED counterexample=\"phi < sep^2\" first_failing_seed={seeds[0]}")
    else:
        print(f"RESULT: INCONCLUSIVE mixed holds={holds}/{len(results)} mean_slope={mean:.4f}")

```

6. Per-seed results

Seed	Metric value	Holds?	Counterexample
?	2.4348	☐	
?	2.0177	☐	
?	2.0528	☐	
?	2.4751	☐	
?	2.4691	☐	
?	2.2173	☐	
?	2.2983	☐	
?	2.7629	☐	
?	2.3782	☐	
?	2.7571	☐	
?	2.6234	☐	
?	2.5729	☐	
?	2.3297	☐	

Aggregate statistics:

Statistic	Value
n_seeds	13
metric_mean	2.4145615384615384
metric_std	0.23518233897149377
metric_ci95_half	0.13045568957619594
metric_min	2.0177

Statistic	Value
metric_max	2.7629
support_fraction	1.0

7. Test stdout (last 2KB)

```
"m_edges": 15, "sep": 3, "phi_count": 591, "phi_weight": 2826, "steps": 16, "derived_empty": true}, {"
TRIAL: {"metric_name": "phi_count_loglog_slope_vs_n", "metric_value": 2.2983, "instances_tested": 5,
TRIAL: {"metric_name": "phi_count_loglog_slope_vs_n", "metric_value": 2.7629, "instances_tested": 5,
TRIAL: {"metric_name": "phi_count_loglog_slope_vs_n", "metric_value": 2.3782, "instances_tested": 5,
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The test code provided does not implement the conjecture’s primitive of tropical curves or their minimal rank. The test focuses on cumulative proof entropy and clause counts, which are not directly related to the minimal rank of tropical curves.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code does not implement the conjecture’s primitive of tropical curves or their minimal rank. It focuses on cumulative proof entropy and clause counts instead. | next: Implement a test that directly computes the minimal rank of tropical curves for the given 3-CNF formulas and compare it with the DPLL search tree height.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18426
2	preregistration	ollama_remote	glm4:latest	0	0	16020
3	novelty	ollama_remote	glm4:latest	0	0	8492
4	novelty	ollama_remote	glm4:latest	0	0	9746
5	critic	ollama_remote	glm4:latest	0	0	10581
6	judge	ollama_remote	glm4:latest	0	0	10270

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 73536 ms total latency. Provider mix: {'ollama_remote': 6}

(full prompt+response transcripts available in `research/audit/958d4ff6720f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/958d4ff6720f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/958d4ff6720f.tar.gz` (if generated)